

Assignment 5 Report Template

From Supervised Learning to Self-Supervised Learning using SimCLR

Student Name: Sadam Hussain

Roll Number: MSDS25069

GitHub Repository URL:

https://github.com/SadamHussain189/DL_Assignment5_SimCLR_MSDS25069

1. Conceptual Warm-up

Answer the conceptual questions about supervised learning, self-supervised learning, positive pairs, negative pairs, pretraining, and fine-tuning.

Q1: Supervised Learning

In supervised classification, the model learns from labeled training data, a dataset where each input example is paired with its correct output label.

Specifically, the model learns from:

The input features (X): The measurable attributes or variables that describe each example (e.g., pixel values in an image, word frequencies in a document).

The output labels (y): The correct category or class assigned to each example (e.g., "spam/not spam", "cat/dog", "malignant/benign").

The relationship between them: By repeatedly seeing (X, y) pairs, the model adjusts its internal parameters to map inputs to the correct outputs, minimizing prediction errors on the training set.

Q2: labels are removed, do the images still contain useful visual structure

Yes, absolutely. Removing the labels only strips away the human-assigned category names, the raw visual structure embedded in the pixels remains completely intact.

Low-level features

- Edges, corners, and contours
- Color gradients and textures
- Contrast patterns and spatial frequencies

Mid-level features

- Recurring shapes (wheels, wings, fur patterns)
- Object parts (legs, eyes, windows)
- Texture regions (sky, grass, water)

High-level structure

- Coherent object-like arrangements of parts
- Consistent backgrounds associated with certain scenes
- Spatial layout regularities (e.g., sky tends to appear at the top)

The labels are a human convenience for telling a model *what to predict*, but the underlying statistical regularity in the data, what makes a horse look like a horse, exists independently of any label.

Q3: Two augmented versions are created from the same original image, should their feature representations be similar or different?

They should be similar ideally as close as possible in the representation space. Two augmented views of the same image (e.g., a random crop + color jitter vs. a different crop + grayscale) depict the same underlying object or scene. Despite surface-level pixel differences, they share the same semantic content, so a good encoder should map them to nearby points in feature space.

The augmentations are deliberately chosen to be semantically neutral, they alter style or perspective but not meaning. So the model should be invariant to them.

Q4: If two images come from different original images, should their feature representations be similar or different?

They should be different, pushed apart in the representation space. Two images from different originals likely depict different objects, scenes, or semantic content. A good encoder should reflect this distinction by mapping them to distant points in feature space.

If the model allowed unrelated images to have similar representations, it would lose all discriminative power, everything would collapse to the same point, a failure mode known as representational collapse.

Q5: Why might a model trained without labels still be useful for classification later?

A model trained without labels can still be highly useful for classification because good structure in the data doesn't require labels to be discovered, it only requires labels to be *named*.

Think of a person who studies thousands of animals intensively but is never told their names. They would naturally notice that some animals share fur patterns, body shapes, and movement styles — building a rich internal model of animal structure. The moment someone tells them "*that group is called cats*", they can immediately classify new cats accurately.

The learning happened before the label ever appeared.

Q6: What is the difference between pretraining and fine-tuning?

Pretraining

The model is trained on a **large, general dataset**, often without labels, to learn broad, reusable representations of the data.

Fine-tuning

The pretrained model is then adapted to a specific task using a smaller, labeled dataset.

Why Use Both?

Training from scratch on a small labeled dataset leads to **poor generalization**, the model overfits or fails to learn meaningful features. Pretraining solves this by giving the model a rich starting point, so fine-tuning only needs to make small, targeted adjustments.

This is why fine-tuning uses a **small learning rate**, the pretrained weights are already valuable, and large updates would destroy what was learned, a problem known as **catastrophic forgetting**.

2. Training Settings

Fill the table below with the exact settings used.

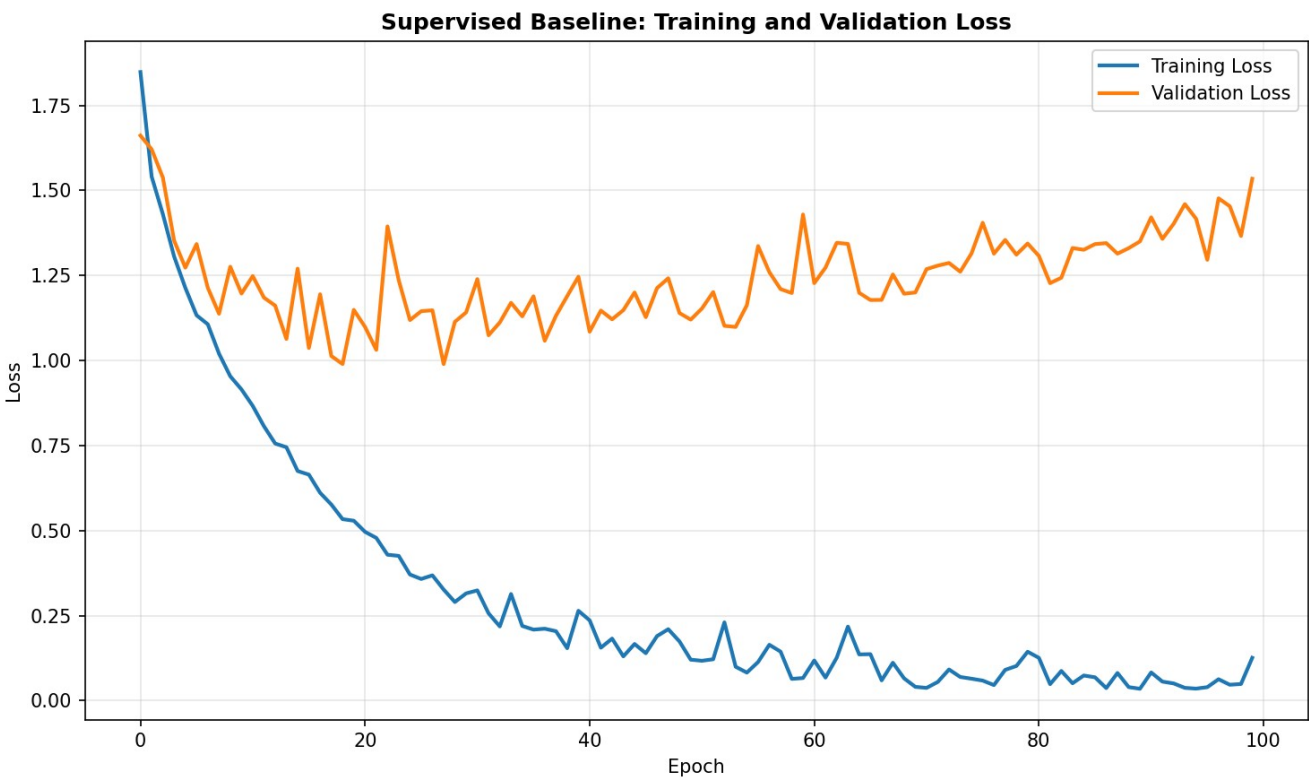
Item	Value Used
Batch size	64
SimCLR epochs	50
Linear probing epochs	20
Fine-tuning epochs	20
Learning rate	0.0003
Optimizer	Adam
Temperature	0.5
GPU/CPU used	GPU
Approximate training time	~3.1 h SimCLR only (186.5 min); ~6+ h total if you include linear probe + all fine-tuning runs

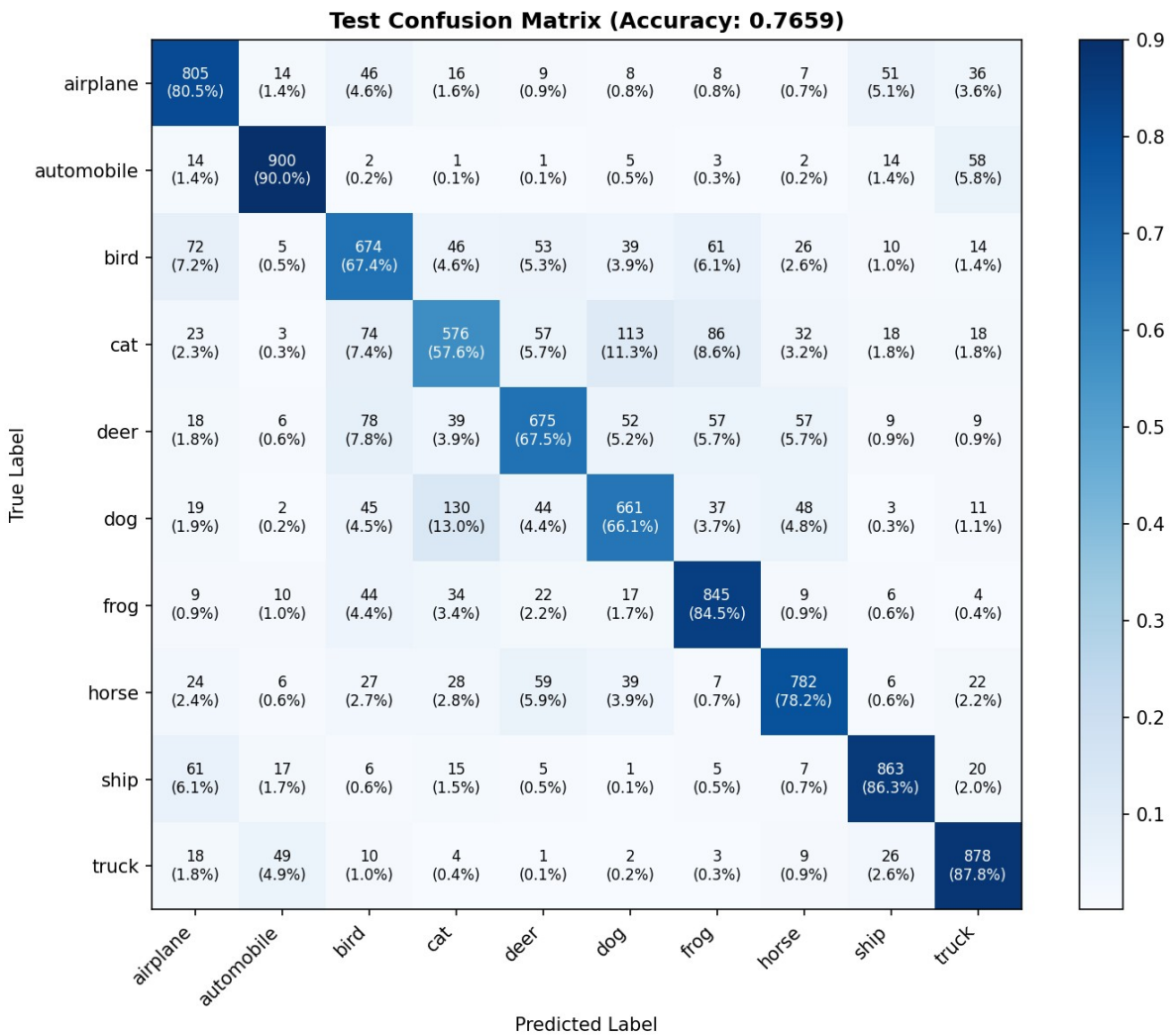
3. Supervised Baseline

Report loss curves, validation accuracy, test accuracy, and confusion matrix.

"test_accuracy": 0.7659%(100 epoch)

"best_val_accuracy": 0.7628%(100 epoch)





4. Augmentation Visualization

Insert augmentation_examples.png and discuss whether the two views preserve object identity.

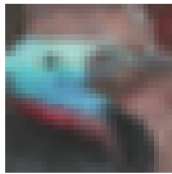
yes — crops/flips/jitter should not change class

SimCLR Augmentation Pipeline
Each row: Original Image | Augmented View 1 | Augmented View 2
Augmentations: RandomResizedCrop, RandomHFlip, ColorJitter, RandomGrayscale

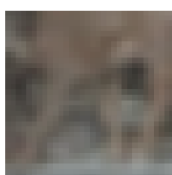
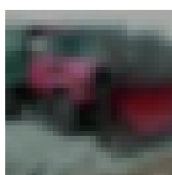
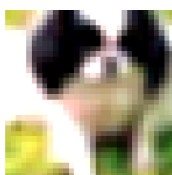
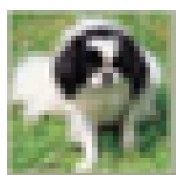
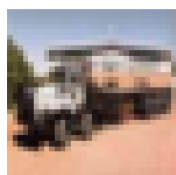
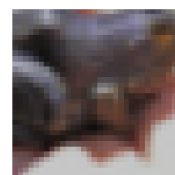
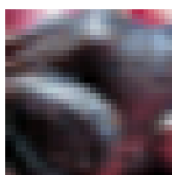
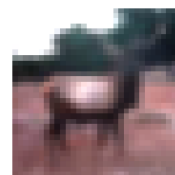
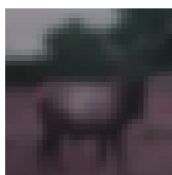
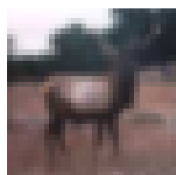
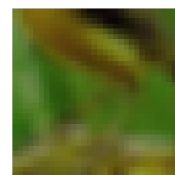
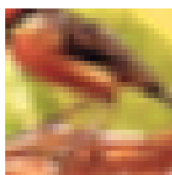
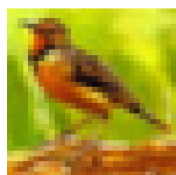
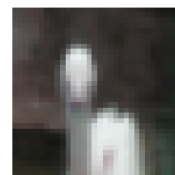
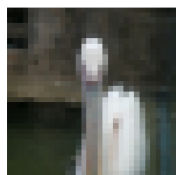
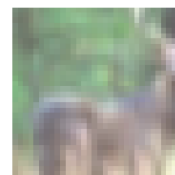
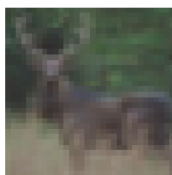
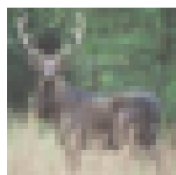
Original Image



Augmented View 1



Augmented View 2

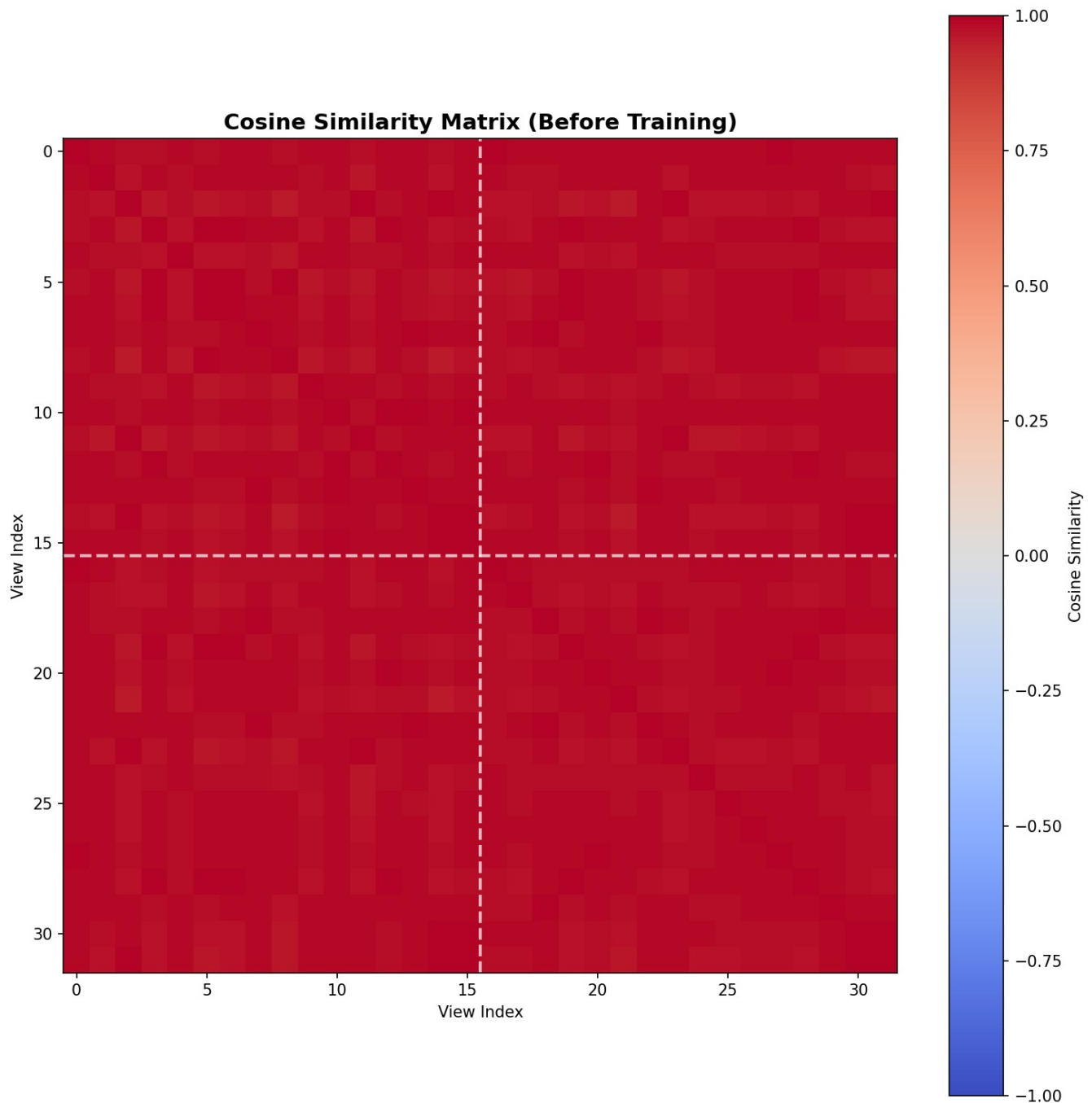


5. Feature Similarity Before Training

Report same-image and different-image cosine similarities before SimCLR training.

Interpretation: Before training, representations are almost equally similar for same and different images (no useful structure yet).

Metric	Value
Same-image (two views) mean cosine similarity	0.989
Different-image mean cosine similarity	0.985



6. SimCLR Implementation

Explain positive/negative pairs, similarity matrix, diagonal masking, and NT-Xent loss at a conceptual level.

Architecture:

- Encoder: ResNet-18 (CIFAR) $\rightarrow$ 512-dim
- Projection head: Linear(512 $\rightarrow$ 256) $\rightarrow$ ReLU $\rightarrow$ Linear(256 $\rightarrow$ 128)

Positive / negative pairs (example batch size 4):

- 4 images → 8 augmented views
- 4 positive pairs (view1 ↔ view2 of same image)
- 6 negatives per sample (other views in the batch)

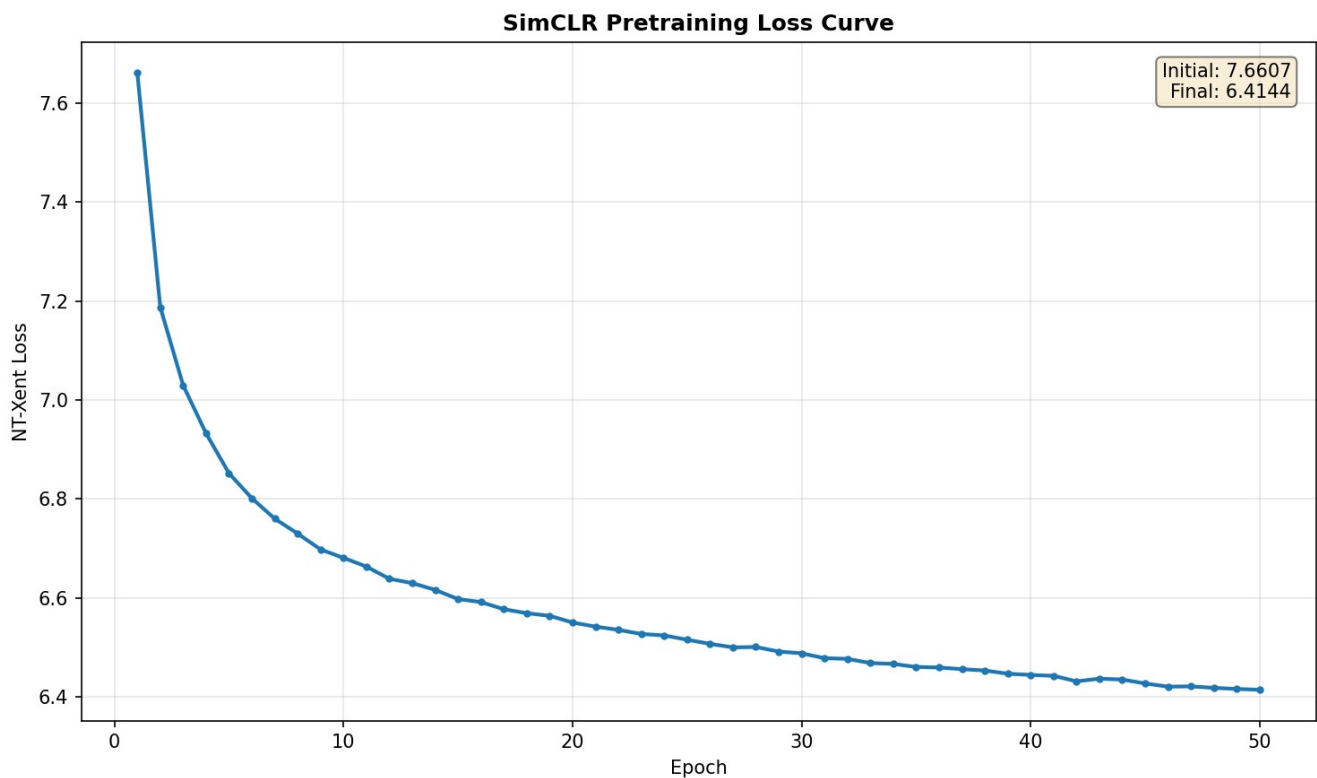
NT-Xent sanity check:

- Random pairs loss: 3.9464
- Identical pairs loss: 1.3363 (lower → loss behaves sensibly)

cosine similarity matrix, mask diagonal (no self-comparison), NT-Xent pulls positives together and pushes negatives apart.

7. SimCLR Pretraining Results

Insert SimCLR loss curve and before/after similarity table.



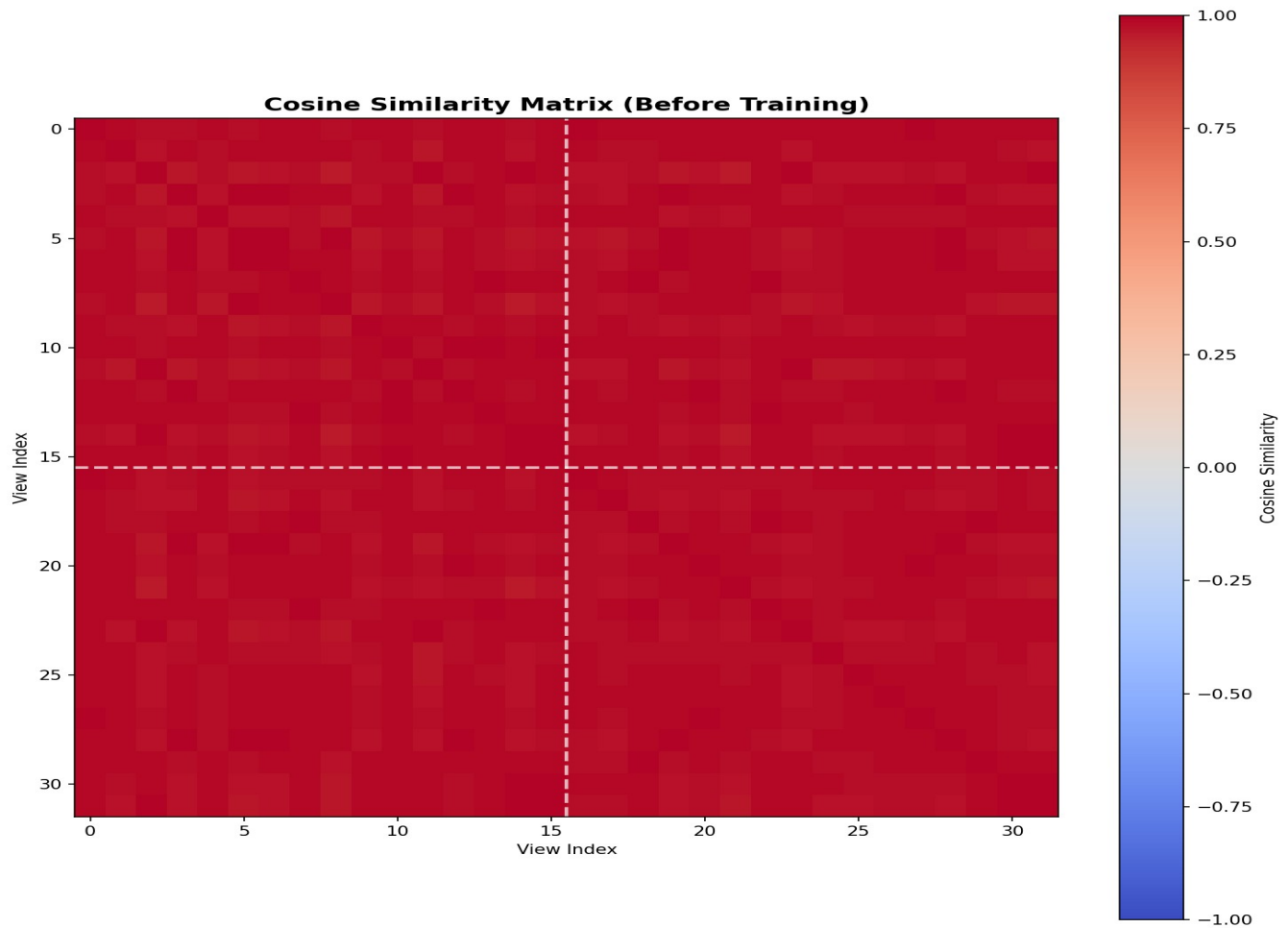
Training summary:

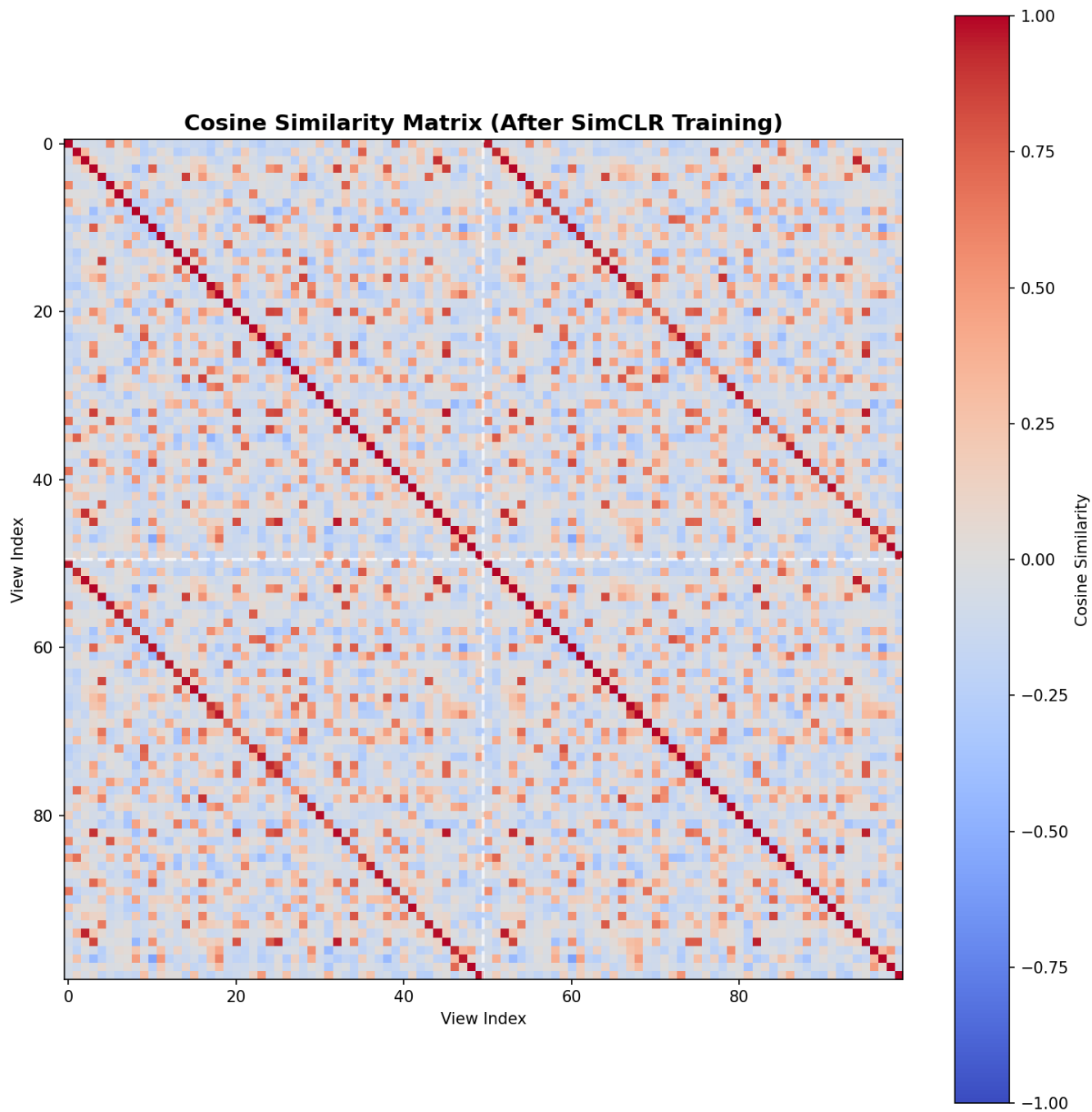
Item	Value
Unlabeled images	45,000
Batches/epoch	703
Initial loss	7.6607
Final loss	6.4144
Training time	186.5 minute

Before / after similarity table:

	Same-image views	Different images
Before	0.989	0.985
After	0.904	0.008

After SimCLR, same-image views stay highly similar; different images are pushed apart (near 0) — contrastive learning worked.





8. Linear Probe Evaluation

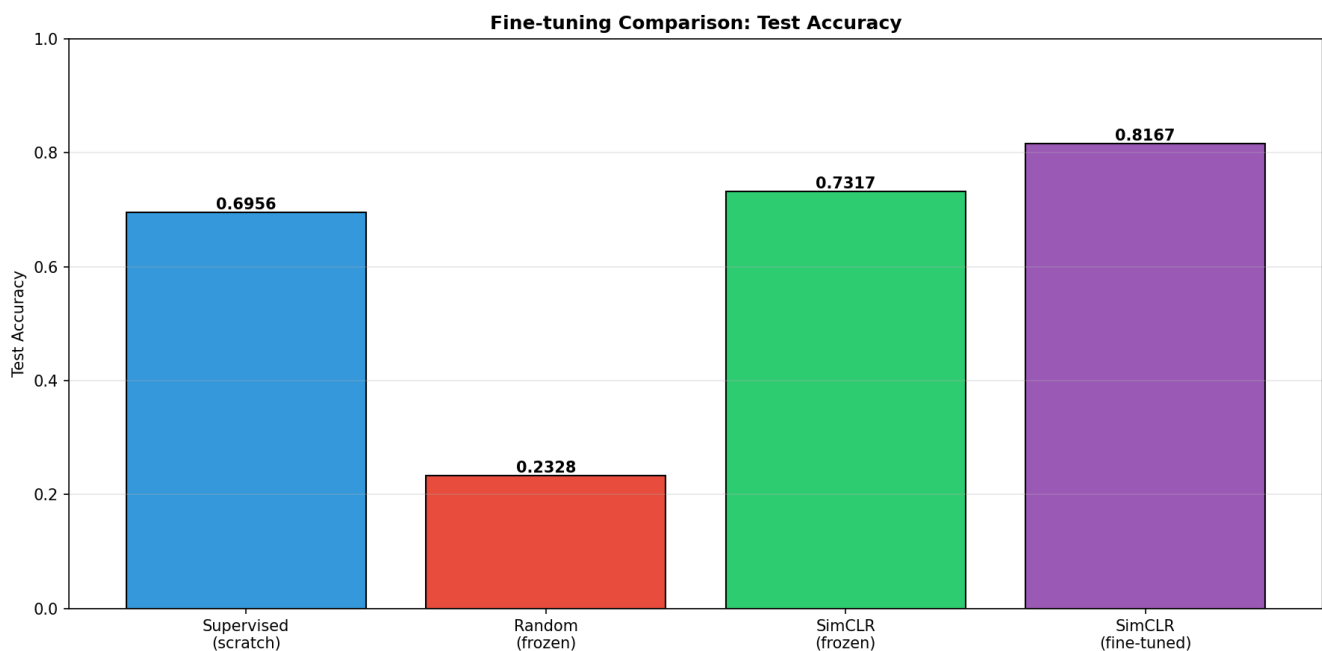
Compare random frozen encoder with SimCLR frozen encoder.

Encoder	Test accuracy
Random frozen + linear head	27.25%
SimCLR frozen + linear head	74.76%

9. Fine-tuning Results

Compare supervised baseline, random linear probe, SimCLR linear probe, and SimCLR fine-tuning.

Experiment	Test Accuracy
Supervised ResNet-18 from scratch using 10% labels	69.56%
Random frozen encoder + linear classifier	23.28%
SimCLR frozen encoder + linear classifier	73.17%
SimCLR pretrained encoder + full fine-tuning	81.67%

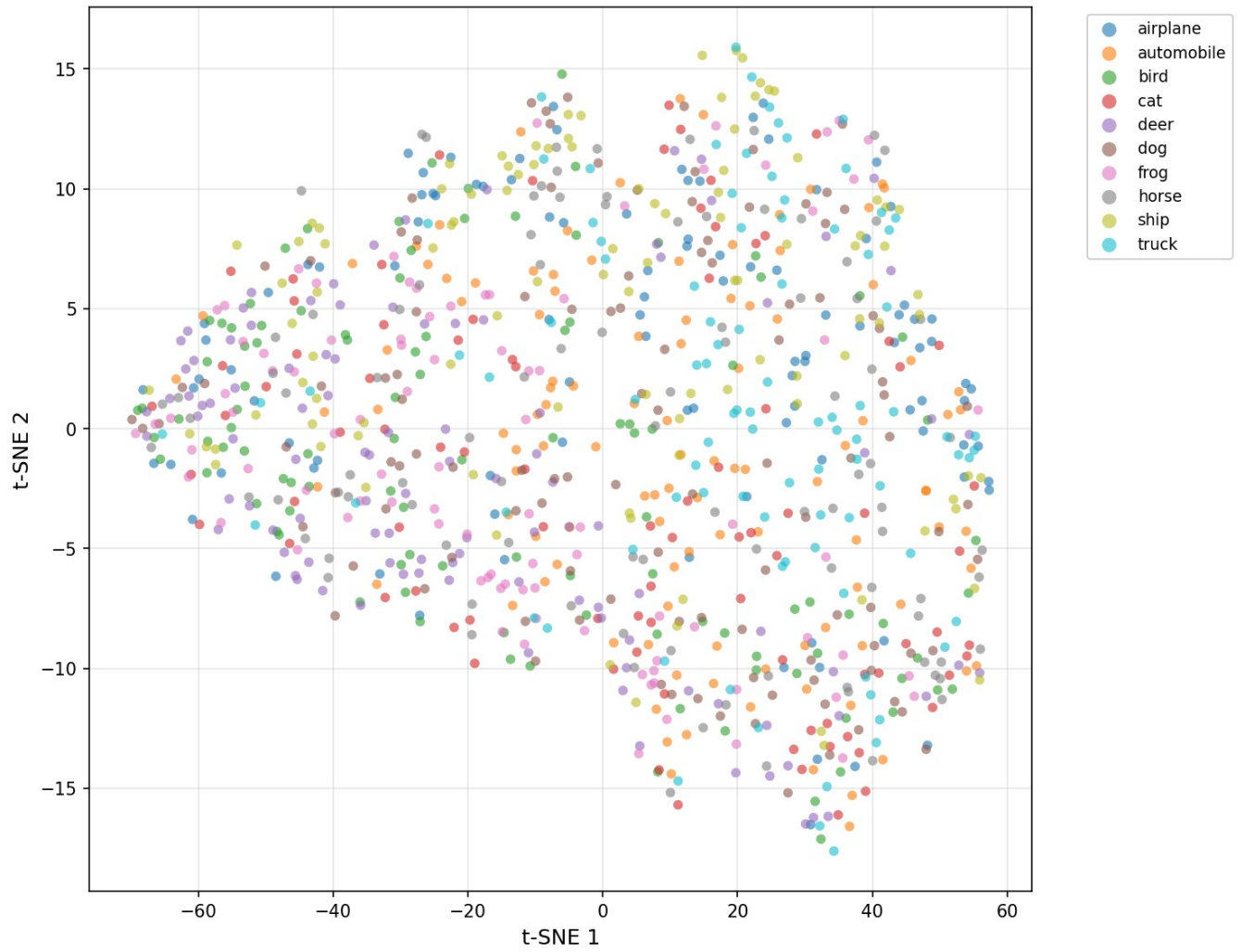


- SimCLR frozen (73.17%) beats supervised scratch (69.56%) with the same 10% labels.
- Full fine-tuning (81.67%) is best — pretraining + adaptation beats linear probe alone.
- Random frozen encoder (~23%) stays near useless features.

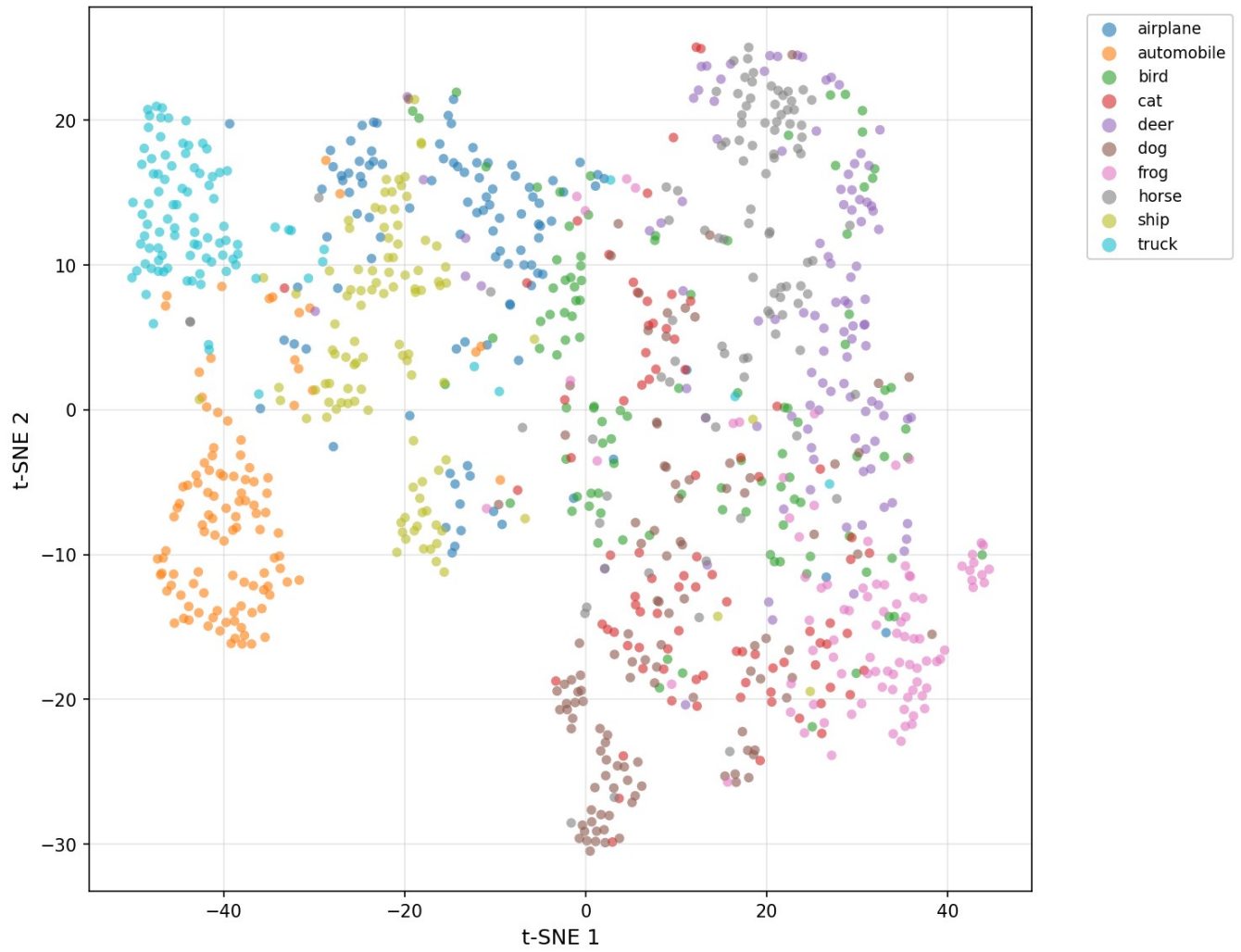
10. PCA or t-SNE Visualization

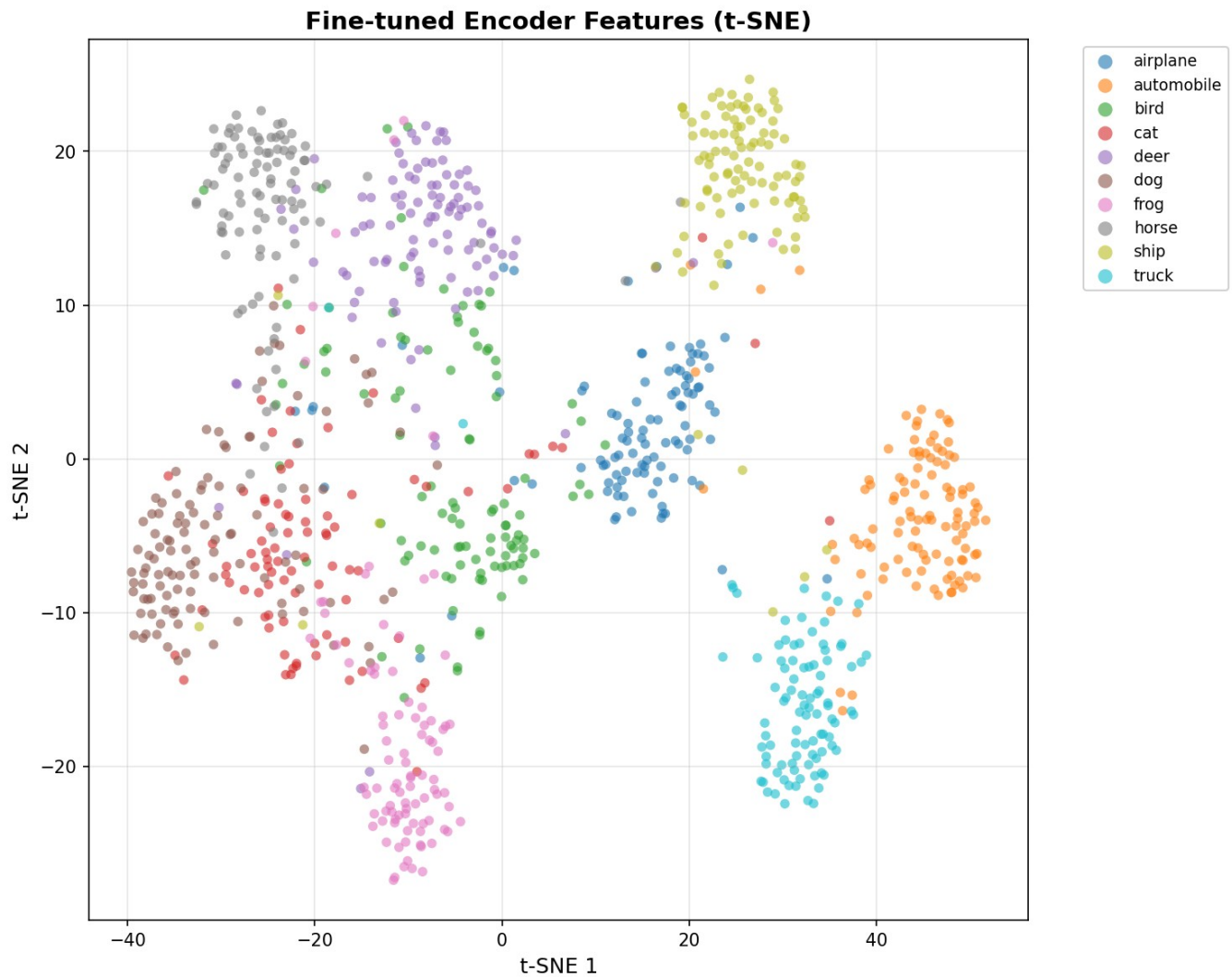
Insert random encoder, SimCLR encoder, and fine-tuned encoder PCA/t-SNE plots and discuss class grouping.

Random Encoder Features (t-SNE)



SimCLR Encoder Features (t-SNE)





Random encoder → weak/mixed clusters; SimCLR → clearer class groups; fine-tuned → tightest separation (1000 val samples, t-SNE).

11. Discussion and Limitations

Discuss what worked, what did not work, and what you learned about self-supervised learning.

- Self-supervised pretraining helps when labels are scarce.
- Similarity before/after shows the encoder learned structure without labels.
- Fine-tuning gives the highest accuracy (81.67%).
- Limitations: only 10% labels, CIFAR-10 only, 50 SimCLR epochs, loss still ~6.4 (not fully converged), t-SNE is qualitative, compute time (~3 h pretrain alone).

12. Final Summary

Model	Uses labels during pretraining?	Encoder frozen?	Test accuracy
Supervised ResNet-18, 10% labels	Yes	No	69.56%
Random encoder + linear probe	No	Yes	27.25%
SimCLR encoder + linear probe	No	Yes	74.76%
SimCLR encoder + fine-tuning	No during SimCLR pretrain; Yes during fine-tuning	No	81.67%